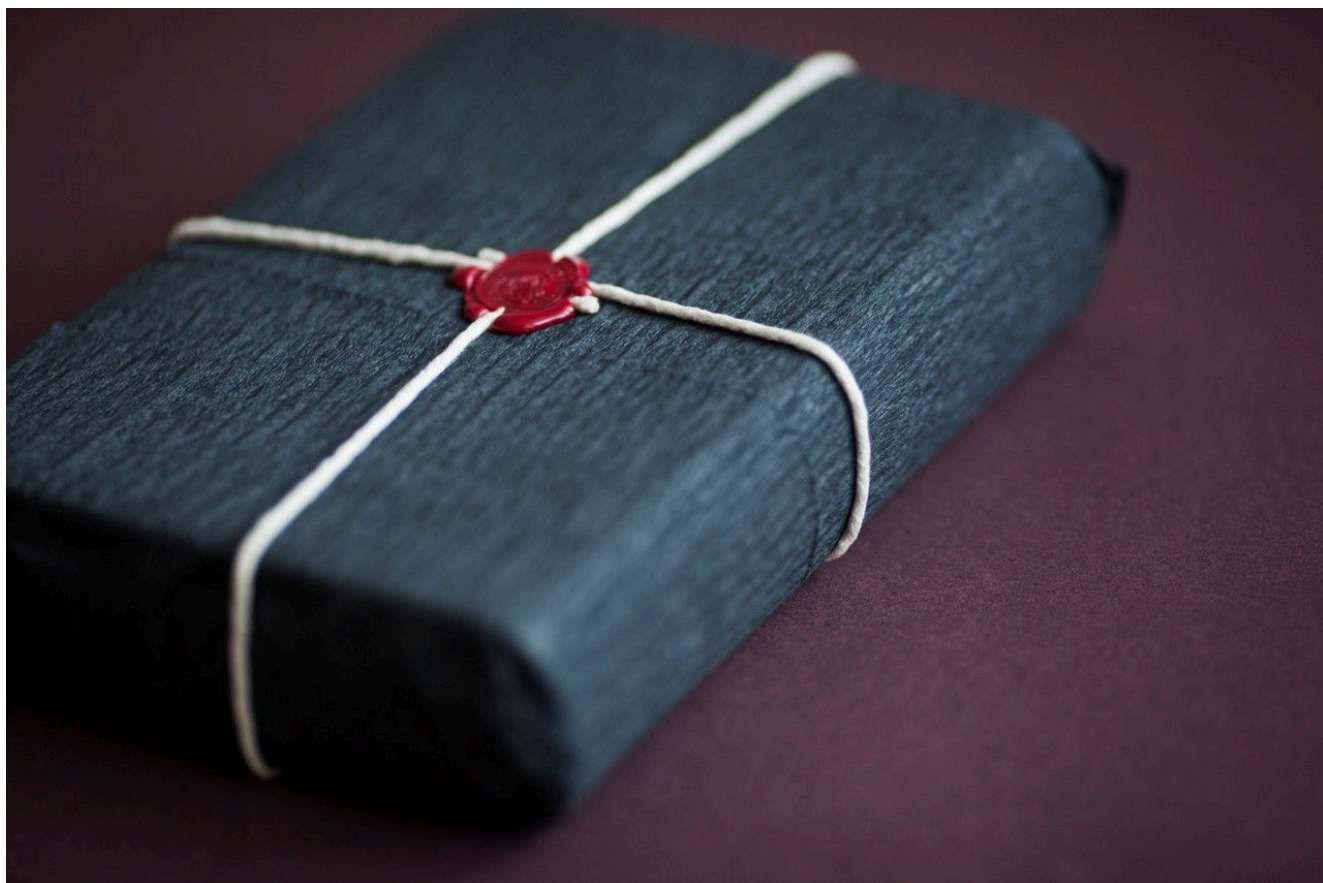


2022年6月15日 / #PYTHON

如何构建你的第一个 Python 包



Chengjun.L



作者： Jason Dsouza

几个月前，我决定发布 Caer，一个可用于 Python 的计算机视觉包。我发现这个过程是非常痛苦的。你可能猜到原因——文档很少而且很混乱，缺乏好的教程，等等。

所以我决定写这篇文章，希望它能帮助那些正在努力做这件事的人。我们将建立一个非常简单的模块，全世界的任何人都可以使用它。

这个模块的内容遵循一个非常基本的结构，总共有四个 Python 文件，每个文件中都有一个方法。这将非常简单。

```
base-verysimplemodule --> Base
└─ verysimplemodule --> Actual Module
    │
    │ └─ extras
    │   │
    │   │ └─ multiply.py
    │   │ └─ divide.py
    │ └─ add.py
    └─ subtract.py
```

你会注意到，我有一个叫作 `verysimplemodule` 的文件夹，里面有两个 Python 文件 `add.py` 和 `subtract.py`。还有一个叫 `extras` 的文件夹（包含 `multiply.py` 和 `divide.py`）。这个文件夹将构成我们的 Python 模块的基础。

`__init__.s`

在每个 Python 包中，你都会发现一个 `__init__.py` 文件。这个文件将告诉 Python 将目录作为模块（或子模块）处理。

学习编程 — 3,000 小时免费课程

一个典型的 `__init__.py` 文件有如下格式：

```
from file import method
```

'method' 是一个函数，存在于一个名为 'file.py' 的文件中

在 Python 中构建包时，你需要在包的每个子目录中添加一个 `__init__.py` 文件。这些子目录是包的子模块。

对于我们的例子，我们将把 `__init__.py` 文件添加到“实际模块”目录 `verysimplemodule` 中，像这样：

```
from add import add  
from subtract import subtract
```

我们将对 `extras` 文件夹做同样的处理，像这样：

```
from multiply import multiply  
from divide import divide
```

一旦完成了这些，我们就基本完成了一半的工作！

如何设置 `setup.py`

在 `base-verysimplemodule` 文件夹中（和我们的模块 `verysimplemodule` 在同一目录下），我们需要添加一个 `setup.py` 文件。如果你打算构建实际的模块，这个文件是必不可少的。

学习编程 — 3,000 小时免费课程

你可能使用 `setup_my_very_awesome_python_package.py` 或 `python_package_setup.py`，但通常最好的做法是坚持使用 `setup.py`。

`setup.py` 文件将包含关于你的软件包的信息，特别是软件包的名称、版本、平台依赖性和其他很多信息。

在这个例子中，我们不需要高级的元信息，所以下面的代码应该适合你构建的大多数软件包。

```
from setuptools import setup, find_packages

VERSION = '0.0.1'
DESCRIPTION = 'My first Python package'
LONG_DESCRIPTION = 'My first Python package with a slightly longer description'

# 配置
setup(
    # 名称必须匹配文件名 'verysimplemodule'
    name="verysimplemodule",
    version=VERSION,
    author="Jason Dsouza",
    author_email="<youremail@email.com>",
    description=DESCRIPTION,
    long_description=LONG_DESCRIPTION,
    packages=find_packages(),
    install_requires=[], # add any additional packages that
    # 需要和你的包一起安装，例如：'caer'

    keywords=['python', 'first package'],
    classifiers= [
        "Development Status :: 3 - Alpha",
        "Intended Audience :: Education",
        "Programming Language :: Python :: 2",
        "Programming Language :: Python :: 3",
        "Operating System :: MacOS :: MacOS X",
        "Operating System :: Microsoft :: Windows",
```

完成这些后，我们接下来要做的就是与 `base-verysimplemodule` 相同的目录下运行以下命令。

```
python setup.py sdist bdist_wheel
```

这将构建所有 Python 需要的必要软件包。`sdist` 和 `bdist_wheel` 命令将创建一个源码分布和一个轮子，你可以随后上传到 PyPi。

PyPi——我们来了！

PyPi 是官方的 Python 仓库，所有的 Python 包都存放在这里。你可以把它看作是 Python 软件包的 GitHub。

为了让全世界的人都能看到你的 Python 软件包，你需要有一个 PyPi 的账户。

完成这些，我们就可以在 PyPi 上上传我们的软件包了。还记得我们运行 `python setup.py` 时建立的源代码分布和轮子吗？好吧，这些才是真正要上传到 PyPi 的东西。

但在这之前，如果你还没有安装 `twine`，你需要安装它，只需运行 `pip install twine` 就可以安装。

如何上传你的软件包到 PyPi

假设你已经安装了 `twine`，继续运行：

```
twine upload dist/*
```

学习编程 — 3,000 小时免费课程

输入这些信息即可。

现在，如果你已经完全按照本教程的要求做了，你可能会得到一个错误，类似于**仓库已经存在**。

这通常是因为你的包的名字和一个已经存在的包的名字冲突了。换句话说，你需要更改你的包的名字——别人已经用了这个名字。

就是这些了！

接下来你可以自豪地 `pip install` 你的模块，请打开终端并运行：

```
pip install <package_name>

# 在你的例子中，应该是
pip install verysimplemodule
```

看看 Python 如何从先前生成的二进制文件中快速地安装你的软件包。

打开一个 Python 交互式 shell 并尝试导入你的软件包。

```
>> import verysimplemodule as vsm

>> vsm.add(2,5)
7
>> vsm.subtract(5,4)
1
```

```
>> import verysimplemodule as vsm

>> vsm.extras.divide(4,2)
2
>> vsm.extras.multiple(5,3)
15
```

就这么简单。

恭喜你！你刚刚建立了你的第一个 Python 包。这非常简单，你的包现在可以被全世界的任何人下载了（当然，他们需要有 Python）。

下一步是什么？

测试 PyPi

我们在本教程中使用的包是一个极其简单的模块——加、减、乘、除的基本数学运算。把它们直接上传到 PyPi 上是没有意义的，尤其是你是第一次尝试这个。

我们很幸运，有 [Test PyPi](#)，一个独立的 PyPi 实例，你可以在那里测试和实验你的软件包（你需要在平台上注册一个独立的账户）。

你上传到 Test PyPi 的过程几乎是一样的，只有一些小的变化。

```
# 以下命令将上传软件包到 Test PyPi
# 你需要提供你的 Test PyPi 证书

twine upload --repository testpypi dist/*
```

```
pip install --index-url "https://test.pypi.org/simple/<package_name>"
```

高级元信息

我们在 `setup.py` 文件中使用的元信息是非常基本的。你可以添加额外的信息，如多个维护者（如果有的话）、作者的电子邮件、许可证信息和一大堆其他数据。

如果你打算这样做，[这篇文章](#)将对你特别有帮助。

看看其他软件仓库

看看其他软件仓库是如何构建软件包的，这对你来说是非常有用的。

在构建 [Caer](#) 时，我会不断地看 [Numpy](#) 和 [Sonnet](#) 是如何构建的。如果你打算构建更高级的软件包，我建议你看看这三个软件仓库。



Chengjun.L

阅读 [更多文章](#)。

在 freeCodeCamp 免费学习编程。freeCodeCamp 的开源课程已帮助 40,000 多人获得开发者工作。

[开始学习](#)

学习编程 — 3,000 小时免费课程

所有给 freeCodeCamp 的捐款都将用于我们的教育项目，购买服务器和其他服务，以及聘用员工。

你可以[点击此处免税捐款](#)。

Trending Books and Handbooks

about:blank 是什么意思	打开 .dat 文件	Node 最新版本
反恶意软件服务	Windows10 产品密钥	Git 切换分支
AppData 文件夹	Windows 10 屏幕亮度	JSON 注释
MongoDB Atlas 教程	Python 字符串转数字	Git 命令
更新 NPM 依赖	谷歌恐龙游戏	CSS 使用 SVG 图片
Python 获取时间	Git Clone 指定分支	JS 字符串反转
React 个人作品网站	媒体查询范围	forEach 遍历数组
撤销 Git Add	OSI 七层网络	Event Loop 执行顺序
CMD 删除文件	Git 删除分支	HTML 表格代码
Nano 怎么保存退出	HTML5 模板	学习编程

移动应用



我们的慈善机构

[简介](#) [校友网络](#) [开源](#) [商店](#) [支持](#) [赞助商](#) [学术诚信](#) [行为规范](#) [隐私条例](#) [服务条款](#)
[版权条例](#)